WG02: The CoreProcessScheduler

Amlal El Mahrouss
NeKernel.org

December 7, 2025

1 Introduction

The CoreProcessScheduler governs how the scheduling backend and policy of the kernel works, It is the common gateway for schedulers inside NeKernel based systems.

2 Abstract

The CoreProcessScheduler (now referred as CPS) serves as the foundation between the scheduler backend and kernel. It takes care of process life-cycle management, team-based process grouping, and affinity-based CPU based allocation to mention the least.

2.1 The Affinity System

Processes are given CPU time affinity hints using an affinity kind type, these hints help the scheduler run or adjust the current process.

2.2 Sample Code #1

The following sample is C++ code. The smaller the value, the more critical the process.

```
enum class AffinityKind : Int32 {
  kRealTime     = 100,
  kVeryHigh     = 150,
  kHigh         = 200,
  kStandard     = 1000,
  kLowUsage     = 1500,
  kVeryLowUsage = 2000,
};
```

2.3 The Team System

The team system holds process metadata for the backend scheduler to run on. It holds methods and fields for backend specific operations. One implementation of such team is the UserProcessTeam object inside NeKernel.

2.4 Sample Code #2

The following sample is used to hold team metadata. This is part of the NeKernel source tree.

```
class UserProcessTeam final {
 public:
  explicit UserProcessTeam();
  ~UserProcessTeam() = default;

  NE_COPY_DEFAULT(UserProcessTeam)

  Array<USER_PROCESS, kSchedProcessLimitPerTeam>& AsArray();
  Ref<USER_PROCESS>&                              AsRef();
  ProcessID&                                      Id() noexcept
    ;

 public:
  USER_PROCESS_ARRAY mProcessList;
  USER_PROCESS_REF   mCurrentProcess;
  ProcessID          mTeamId{0};
  ProcessID          mProcessCur{0};
};
```

2.5 The Process Image System

The process image container is a design pattern made to contain process data and metadata, its purpose comes from the lack of mainstream operating systems of such ability to hold metadata.

This approach helps separate concerns and give modularity to the system, as the image and process structure are not mixed together.

2.6 Sample Code #3

The following sample is a C++ container used to hold process data and metadata. This is part of the NeKernel source tree.

```
using ImagePtr = void*;

struct PROCESS_IMAGE final {
  explicit PROCESS_IMAGE() = default;

 private:
  friend USER_PROCESS;
```

```
  friend KERNEL_TASK;

  friend class UserProcessScheduler;

  ImagePtr fCode;
  ImagePtr fBlob;

 public:
  Bool HasCode() const { return this->fCode != nullptr; }

  Bool HasImage() const { return this->fBlob != nullptr; }

  ErrorOr<ImagePtr> LeakImage() {
    if (this->fCode) {
      return ErrorOr<ImagePtr>{this->fCode};
    }

    return ErrorOr<ImagePtr>{kErrorInvalidData};
  }

  ErrorOr<ImagePtr> LeakBlob() {
    if (this->fBlob) {
      return ErrorOr<ImagePtr>{this->fBlob};
    }

    return ErrorOr<ImagePtr>{kErrorInvalidData};
  }
};
```

3 Conclusion

The CoreProcessScheduler is a piece of systems design with robust design and useful cases, although useful in desktop/server cases, It may not be suited for every other tasks.

And while one scheduler backend (such as the UserProcessScheduler) takes care of user process scheduling and fairness, the CoreProcessScheduler takes care of the foundation for those systems.

3.1 References

CoreProcessScheduler: Link

NeKernel.org: Link

Round Robin (OSTEP): Link

Processor Affinity: Link